Python Chapter 5 Functions Activities

Contents

Python Chapter 5 Functions Activities 1
Online Tutorials 1
Functions 2
Tutorial 5.1: Miles to Kilometers 3
Scope and Lifetime 4
  Local Variables 4
Keyword Arguments 4
Default Arguments 5
Functions with Unknown Number of Arguments 5
Tutorial 5.2: Argument Soccer Mayhem 6
Docstrings 9
Input Validation with Boolean Functions 10
Try Except 11
Tutorial 5.3: Fahrenheit to Celsius 12
Tutorial 5.4: Fahrenheit to Celsius Try Except 13
lambda Function 14
  Assignment Submission 14

Time required: 90 minutes

Online Tutorials

Go through the following tutorials.

- LearnPython.org Functions
- Python Functions

Functions

Functions in Python are blocks of organized, reusable code that perform a single, related action.

Definition: A function is defined using the **def** keyword, followed by the function name and parentheses **()**. The code block that the function runs is indented.

- Use the **def** keyword, followed by the function name and parentheses.
- Example: **def greet():**

Function Body: Indent the code within the function to define its body.

```
# Function definition
def my_function():
    # Function body
    print("Hello from a function")
```

Calling: To run the code inside the function, you need to call it by its name followed by parentheses **()**

```
my_function()
```

Parameters: Functions can take parameters, which are values you pass into the function. These parameters act as variables inside the function.

Arguments: Provide values for the parameters when calling the function.

Example: **greet("Alice")**

```
def greet(name):
    print("Hello, " + name)

greet("Alice")
```

Return Values: Functions can also return values using the **return** keyword. The value returned by the function can be stored in a variable when the function is called.

```
def square(x):
    return x * x

result = square(3)
print(result)
# Output: 7
```

Tutorial 5.1: Miles to Kilometers

Create a Python program caled **miles_to_kilometers.py**

This program uses a function with a parameter and a return value. It uses a main() function to start the program.

```python
"""
    Name: miles_to_kilometers.py
    Author:
    Created:
    Purpose: Convert from miles to kilometers
"""


def miles_to_kilometers(miles):
    """
    Convert miles to kilometers.
    :param miles: Distance in miles
    :return: Equivalent distance in kilometers
    """
    kilometers = miles * 1.60934
    return kilometers


def main():
    # Get user input for miles
    miles_input = float(input("Enter distance in miles: "))

    # Call the function to convert miles to kilometers
    kilometers_result = miles_to_kilometers(miles_input)

    # Display the result
    print(f"{miles_input} miles is: {kilometers_result:.2f} kilometers.")


# Run the program
if __name__ == "__main__":
    main()
```

Example run:

```
Enter distance in miles: 60
60.0 miles is: 96.56 kilometers.
```

Scope and Lifetime

Scope refers to visibility. When discussing the scope of a variable, it refers to the places in a program where a variable is visible and can be changed. Lifetime refers to how long a variable exists in memory. Lifetime is closely related to scope.

In general, the scope of a variable is the code block in which it is created and all the code blocks that are nested within that code block at a deeper indent level.

Local Variables

What happens in the function, stays in the function.

Let's say we have two functions like the ones below that each use a variable `i`:

```
def function1():
    for i in range(10):
        print(i)

def function2():
    i = 100
    function1()
    print(i)
```

A problem that could arise here is that when we call **function1**, we might mess up the value of **i** in **function2**. In a large program it would be a nightmare trying to make sure that we don't repeat variable names in different functions.

Fortunately, we don't have to worry about this. When a variable is defined inside a function, it is local to that function, which means it does not exist outside that function. Each function can define its own variables and not have to worry about if those variable names are used in other functions.

Keyword Arguments

Python offers another convention for passing arguments, where the meaning of the argument is dictated by its name, not by its position - it's called keyword argument passing.

```
def introduction(first_name, last_name):
```

```
    print(f"Hello, my name is {first_name} {last_name}")

introduction(first_name = "James", last_name = "Bond")
introduction(last_name = "Skywalker", first_name = "Luke")
```

The concept is clear - the values passed to the parameters are preceded by the target parameters' names, followed by the = sign. The position doesn't matter here - each argument's value knows its destination on the basis of the name used.

Default Arguments

It happens at times that a particular parameter's values are in use more often than others. Such arguments may have their default (predefined) values taken into consideration when their corresponding arguments have been omitted.

They say that the most popular English last name is Smith. Let's try to take this into account. The default parameter's value is set using clear and pictorial syntax:

```
def introduction(first_name, last_name=”Smith”):
    print(f"Hello, my name is {first_name} {last_name}")

introduction(first_name = "James")
introduction(last_name = "Skywalker", first_name = "Luke")
```

```
Hello, my name is James Smith
Hello, my name is Luke Skywalker
```

Functions with Unknown Number of Arguments

Python allows functions with an unknown number of arguments. The argument definition uses the * to indicate that any number of arguments can be sent to the function. The function does have to be written to accommodate an unknown number of arguments.

```
def calculate_total(*args):
    total = 0
    for number in args:
        total += number
    print(total)
# function calls
calculate_total(5, 4, 3, 2, 1)
calculate_total(35, 4, 3)
```

Example run:

```
15
42
```

You can combine both specific arguments, and multiple arguments.

```
def the_rest(first, second, third, *args):
    print(f"First: {first}")
    print(f"Second: {second}")
    print(f"Third: {third}")
    print(f"And all the rest... {list(args)}")

the_rest(1, 2, 3, 4, 5)
```

Example run:

```
First: 1
Second: 2
Third: 3
And all the rest... [4, 5]
```

```
def sum(*args):
    result = 0
    for i in args:
        result += i
    return(i)

print(sum(1, 3, 5, 6))
return_sum = sum(100, 80, 33)
print(return_sum)
```

Tutorial 5.2: Argument Soccer Mayhem

Function with default arguments: We define the **player_stats** function with default values for goals, assists, and minutes played. If these values are not provided, they default to 0. Default arguments are useful when you want to make a parameter optional and provide a reasonable default value if the caller doesn't specify one.

Function with keyword arguments: In **player_goals** we specify values for specific statistics using the ****kwargs** parameter. This allows flexibility in specifying the values.

When calling the function, we use keyword arguments to specify values for these parameters. The order in which we provide the keyword arguments doesn't need to match the order of the parameters in the function definition, making it more flexible and readable.

Function with unknown number of arguments: We create a function that accepts an unknown number of arguments using the ***args** parameter. This function will display additional statistics for the player. You can call the function with different numbers of arguments, and it will still work correctly.

```python
"""
 * Name: soccer_stats.py
 * Written by:
 * Written on:
 * Purpose: Demo of 3 different types of arguments
"""


# ----------------------- DEFAULT ARGUMENTS -------------------------------#
def player_stats(player_name, goals=0, assists=0, minutes_played=0):
    # There are default values for some of the arguments
    # If an argument is not supplied, the default values are used
    print(f"Player: {player_name}")
    print(f"Goals: {goals}")
    print(f"Assists: {assists}")
    print(f"Minutes Played: {minutes_played}")


# ---------------- UNKNOWN NUMBER OF ARGUMENTS --------------------------#
def additional_stats(player_name, *args):
    print(f"Player: {player_name}")
    # Loop through however many arguments are supplied
    for stat in args:
        print(stat)
```

```python
# ------------------------ KEYWORD ARGUMENTS -------------------------------#
# Function with keyword arguments
def player_goals(player_name, **kwargs):
    print(f"Player: {player_name}")
    # Display only the arguments that match the keywords
    if 'regular_goals' in kwargs:
        print(f"Regular Goals: {kwargs['regular_goals']}")
    if 'penalty_goals' in kwargs:
        print(f"Penalty Goals: {kwargs['penalty_goals']}")


print("Default arguments")
# Call the function with default arguments
player_stats("Lionel Messi")

print("\nUnknown number of arguments")
# Call the function with various statistics
additional_stats("Neymar", "Yellow Cards: 3", "Red Cards: 1", "Assists: 10")

print("\nKeyword arguments")
# Call the function with keyword arguments
player_goals("Cristiano Ronaldo", regular_goals=20, penalty_goals=5)
```

Example run:

```
Default arguments
Player: Lionel Messi
Goals: 0
Assists: 0
Minutes Played: 0

Unknown number of arguments
Player: Neymar
Yellow Cards: 3
Red Cards: 1
Assists: 10

Keyword arguments
Player: Cristiano Ronaldo
Regular Goals: 20
Penalty Goals: 5
```

Docstrings

A docstring in Python is a string literal that serves as documentation for a module, function, class, or method. It is placed immediately after the definition and is enclosed in triple quotes (either single or double). Docstrings are used to provide information about what the code does, how it should be used, and any relevant details.

1. **Documentation Purpose**: Docstrings are primarily used for documenting code to make it more understandable for developers, including yourself and others who might use or maintain the code.
2. **Location:** Docstrings are placed immediately after the definition. This makes them easily accessible when someone wants to learn about the code's purpose and usage.
3. **Triple Quotes:** Python allows docstrings to be enclosed in either triple single quotes (''') or triple double quotes (""""). Triple quotes are used to allow multi-line docstrings.
4. **Content:** Docstrings typically include information such as a brief description of the code's purpose, explanations of parameters and return values, usage examples, and any special considerations or notes.
5. **Access:** Docstrings can be accessed directly through many IDE's such as Visual Studio Code. Hover your mouse over the function call, the function definition and docstring appear. Many Python libraries use docstrings to in a similar fashion to help the programmer quickly understand how to use the function.

Docstring example:

```
def add(a, b):
    """
    Add two numbers and return the result.

    Parameters:
    a (int or float): The first number.
    b (int or float): The second number.

    Returns:
    int or float: The sum of 'a' and 'b'.
    """
    return a + b
add(1, 45)
```

In VSCode.

```python
def add(a: float, b: float) -> float:
    """
    Add two numbers and return the result

    Parameters:
    a (int or float): The first number.
    b (int or float): The second number.

    Returns:
    int or float: The sum of 'a' and 'b'.
    """
```

```
(function) def add(
    a: float,
    b: float
) -> float
```

Add two numbers and return the result.

Parameters:
a (int or float): The first number.
b (int or float): The second number.

Returns:
int or float: The sum of 'a' and 'b'.

```python
add(1, 45)
```

Input Validation with Boolean Functions

The following code provides a Boolean return value. Instead of catching program exceptions, this function evaluates whether the input fits the range of the data. This function can be combined with the last input function.

```
def main():
    number = int(input("Please enter a positive number: "))
    # while the is_invalid function returns true
    # the loop keeps asking for valid input
    while is_invalid(number):
        print("Try again.")
        number = int(input("Please enter a positive number: "))

    print(f"Success! {number} is a positive number. ")

def is_invalid( number ):
    """While this function returns true, the data is invalid"""
    if number < 1:
        status = True
    else:
        status = False

    return status

main()
```

Example run:

```
Please enter a positive number: -1
Try again.
Please enter a positive number: 2
Success! 2 is a postive number.
```

Try Except

Things go wrong, errors happen, let's learn to handle those exceptional events with try except.

An exception is a signal that a condition has occurred that can't be easily handled using the normal flow-of-control of a Python program. Exceptions are often defined as being "errors" but this is not always the case. All errors in Python are dealt with using exceptions, but not all exceptions are errors.

Go through this short tutorial for a better understanding of exceptions.

- Python Try Except

Basic syntax of exception handling

```
try:
    # Code that might raise an exception
except ExceptionType as e:
    # Code to handle the exception
```

Tutorial 5.3: Fahrenheit to Celsius

Create and test a program named **fahrenheit_to_celsius.py** This program does not have any exception handling.

Note: You can get the ° degree from any number of websites or copy it from this sentence.

```
1  """
2      Name: fahrenheit_to_celsius.py
3      Author:
4      Created:
5      Purpose: Convert Fahrenheit input to Celsius
6  """
7
8  # Get Fahrenheit temperature from user
9  fahrenheit = float(input("Enter a Fahrenheit temperature: "))
10
11 # Calculate the Celsius equivalent
12 celsius = ((fahrenheit - 32.0) * 5.0) / 9.0
13
14 # Display the Celsius temperature
15 print(f"{fahrenheit:.2f}° Fahrenheit is equal to {celsius:.2f}° Celcius.")
```

Example program runs:

```
Enter a Fahrenheit temperature: -40
-40.00° Fahrenheit is equal to -40.00° Celcius.
```

If we execute this code and give it invalid input, it simply fails with an unfriendly error message:

```
Enter a Fahrenheit temperature: Barney
Traceback (most recent call last):
  File "Z:\_WNCC\Python\Assignments\03 Conditional Execution\Chapter.03 Fahrenhe
it to Celsius.py", line 8, in <module>
    fahrenheit = float(input('Enter a Fahrenheit temperature: '))
ValueError: could not convert string to float: 'Barney'
```

There is a conditional execution structure built into Python to handle these types of expected and unexpected errors called “try / except”. The idea of try and except is that you know that some sequence of instruction(s) may have a problem and you want to add some

statements to be executed if an error occurs. These extra statements (the except block) are ignored if there is no error.

You can think of the **try** and **except** feature in Python as an "insurance policy" on a sequence of statements.

Tutorial 5.4: Fahrenheit to Celsius Try Except

We rewrite our temperature converter as follows:

```
"""
    Name: fahrenheit_to_celsius_try.py
    Author:
    Created:
    Purpose: Use try except to handle improper input
"""

# Try to execute the program
try:
    # Get Fahrenheit temperature from user
    fahrenheit = float(input("Enter a Fahrenheit temperature: "))

    # Calculate the Celsius equivalent
    celsius = ((fahrenheit - 32.0) * 5.0) / 9.0

    # Display the Celsius temperature
    print(f"{fahrenheit:.2f}° Fahrenheit is equal to {celsius:.2f}° Celcius.")

# Catch and handle a program exception
except:
    # Display an error message
    print("Please enter a number next time.")
    # raise is for troubleshooting a program that has an exception
    # raise shows the stack trace as if the try except block didn't exist
    # raise
```

Python starts by executing the sequence of statements in the try block. If all goes well, it skips the except block and proceeds. If an exception occurs in the try block, Python jumps out of the try block and executes the sequence of statements in the except block.

```
Enter a Fahrenheit temperature: Barney
Please enter a number next time.
```

Handling an exception with a try statement is called catching an exception. In this example, the except clause prints an error message. In general, catching an exception gives you a chance to fix the problem, or try again, or at least end the program gracefully.

lambda Function

A **lambda** function is an inline, single, or anonymous function which is defined without a name. It is used for simple, short functions, instead of writing out an entire function. Lambda functions are often used in graphical user interface programs.

- A lambda function can take any number of arguments.
- It can only have one expression.
- The result is automatically returned.
- It is defined using the **lambda** keyword.

```
# Filename: lambda_simple.py
# lambda arguments: expression
# argument: a
# expression: a * a
x = lambda a: a * a
# Call and print the lambda function
print(x(5))
```

Example run:

```
25
```

x is passed into the lambda function. "Hello" is return if x > 5, else "bye" is returned.

```
x = lambda x: "hello" if x > 5 else "bye"
print(x(10))
print(x(4))
```

Example run:

```
Hello
Bye
```

Assignment Submission

1. Attach the program files.
2. Attach screenshots showing the successful operation of the program.
3. Submit in Blackboard.